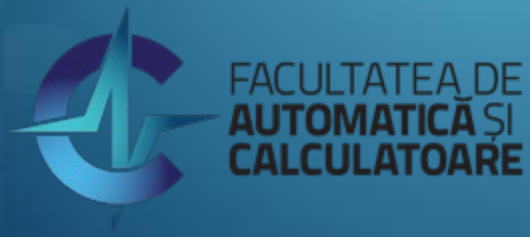


Algoritmi Paraleli și Distribuiți
Algoritmi undă

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





Algoritmi undă

- În dezvoltarea algoritmilor distribuiți apar frecvent anumite tipuri de **probleme generale** pentru **rețele de procese**.
- Rezolvate prin transmitere de mesaje după **o schemă predefinită**, **dependentă de topologie** care asigură participarea **tuturor proceselor**.
- Justifică tratarea lor izolată de alți algoritmi în care aceste scheme pot fi folosite.

Notiuni preliminare - configuratii

- Un **program distribuit** consta dintr-o **colectie de procese** secventiale comunicante
- **Configuratie** = Ansamblul **starilor** tuturor proceselor la un moment dat
- O **operatie** modifica starea unui proces, implicit configuratia
- O executie a programului poate fi **modelata** printr-o secventa de **configuratii**.

- Ex.1 Program distribuit = colectie a doua procese, initial a si b au valoarea 0
- process P1{ int a = 0; a = a+1;} process P2{ int b = 0; b = b+1;}
- configuratia = ansamblul valorilor celor doua variabile modificate de procese → configuratia initiala este <0,0>
- daca operatia primului proces se executa mai intai, configuratia devine <1,0>
- O executie posibila = secventa de configuratii: (<0,0> , <1,0> , <1,1>)

Sisteme de tranziţii

- Un program distribuit poate fi modelat ca un **sistem de tranziții**:
  - Mulțimea tuturor stărilor (**configurațiilor**) posibile ale sistemului
  - Tranzițiile pe care sistemul le poate face între stări
  - Stările din care sistemul poate porni (inițiale)

- Formal, un **sistem de tranziții** este un triplet $S = (C, \rightarrow, I)$
  C este o mulțime de configurații
  $\rightarrow$ este relația de tranziție binară pe C
  I este setul configurațiilor inițiale (o submulțime a lui C)

- O **execuție** a lui S este o secvență maximală
  $E = (\gamma_0, \gamma_1, \gamma_2, \dots)$, unde $\gamma_0 \in I$ și $\gamma_i \rightarrow \gamma_{i+1}$, $pentru\ i \geq 0$.

- O configurație **terminală** γ nu are succesor:
  $\nexists\ \delta$ astfel încât $\gamma \rightarrow \delta$.

Sisteme de tranziţii (2)

- O secvență E este **maximală** dacă:
  - Este infinită sau
  - Sfârșește într-o configurație terminală.
- Configurația δ este **tangibilă** din γ dacă există o secvență $\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_k$ a.î.:

$$\gamma = \gamma_0$$
$$\delta = \gamma_k$$
$$\gamma_i \rightarrow \gamma_{i+1}, pentru\ i = \overline{0, k-1}$$

- Un **sistem distribuit** constă dintr-o colecție de procese și un subsistem de comunicații.
- **Convenții:**
  - pentru sistem: **tranziții** și **configurații**
  - pentru proces: **evenimente (interne / send,receive)** și **stări**
- Unei **execuții** E îi corespunde o **secvență de evenimente** din diferite procese.

Sisteme de tranziţii (3)

- Pentru o execuție E, **relația de ordine cauzală** $<$ este cea mai slabă relație care satisface:
  - dacă $\boldsymbol{a}$ și $\boldsymbol{b}$ sunt două evenimente diferite ale aceluiași proces și $\boldsymbol{a}$ se produce înaintea lui $\boldsymbol{b}$ atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a}$ este un eveniment send, iar $\boldsymbol{b}$ evenimentul receive corespunzător atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{c}$ atunci $\boldsymbol{a} < \boldsymbol{c}$ (< este tranzitivă).
- Dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{a}$ atunci a și b sunt **concurente**.

- O execuție E este **echivalentă** cu F ($E \sim F$) dacă:
  - au aceeași colecție de evenimente (ordinea diferă)
  - evenimentele respectă aceeași relație de ordine cauzală
  - ultima configurație a lui E coincide cu ultima configurație a lui F
- **Obs**: două execuții echivalente pot să nu aibă aceleași configurații.
- Un **calcul** (computation) al unui algoritm distribuit este o clasă de echivalență (sub relația $\sim$) a execuțiilor algoritmului.

Algoritmi undă (Wave algorithms)

- **Schema** de transmitere de mesaje:
  - dependentă de topologie
  - asigură participarea tuturor proceselor
- Algoritmii undă implementează astfel de scheme pentru:
  - difuzarea informației
  - realizarea unei sincronizări globale între procese
  - declanșarea unui eveniment în fiecare proces
  - calculul unei funcții în care fiecare proces participă cu o parte a datelor de intrare
- Proprietăți:
  - **terminare** – fiecare *calcul* este finit
  - **decizie** – fiecare *calcul* conține cel puțin un eveniment de decizie (*decide*)
  - **dependență** – în fiecare *calcul*, fiecare *decide* este precedat cauzal de un eveniment în fiecare proces

Definiții (1)

Calcul (computation) ⇔ **undă** (wave)

Categorii de procese:

Inițiatori (starters) – primul eveniment este unul intern sau un send

Neinițiatori (followers) – primul eveniment este un receive

Clasificare:

- centralizare
  - algoritmi centralizați – un inițiator
  - algoritmi descentralizați – set arbitrar de inițiatori

- topologie
  - inel, arbore, clică, etc.
  - fixă (nu se produc modificări topologice)
  - nedirecționată (canale bidirecționale); excepțiile menționate explicit
  - conectată (există o cale între oricare două procese)

Definiții (2)

- cunoștințe inițiale – exemple:
  - identitatea proprie (nume)
  - identitățile vecinilor
  - **setul direcției (ordinea vecinilor)**

- numărul de decizii (regulă = cel mult o decizie în fiecare proces)
  - un singur proces decide
  - toate decid
  - unele decid

- complexitate
  - număr de mesaje schimbate
  - număr de biți interschimbați
  - timpul necesar pentru un calcul

Noțiuni preliminare
Sensul legăturilor

Complexitatea comunicării într-un algoritm distribuit depinde de topologie, dar și de:

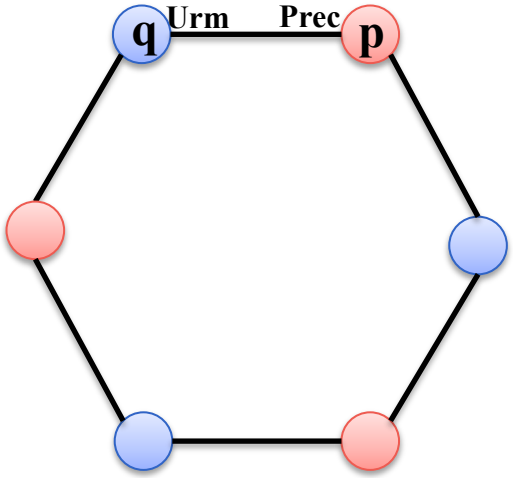
- cunoașterea topologiei la nivelul fiecărui nod (topological awareness)

- setul de direcții (sense of direction)
  - muchiile incidente unui nod sunt etichetate cu direcția spre care conduc în rețea
  - setul de etichete este același pentru fiecare nod
  - mărimea setului depinde de topologie (2 pt. inel, 4 pt. tor, etc)
  - trebuie respectată o **condiție suplimentară de consistență**

Noțiuni preliminare
Sensul legăturilor (2)

Inel:

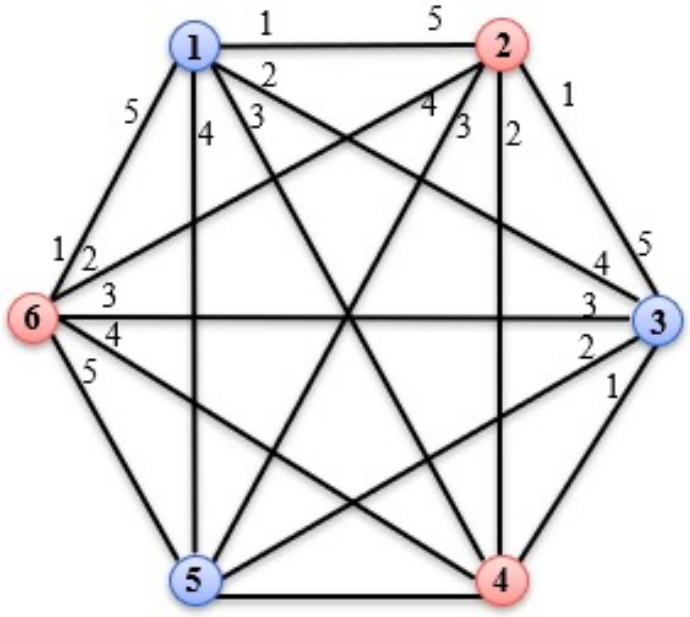
- 2 direcții: *Prec* (precedent) și *Urm* (următor)
- condiția de consistență: **precedentul lui p este q ⇔ următorul lui q este p**



Noțiuni preliminare
Sensul legăturilor (3)

Clică:

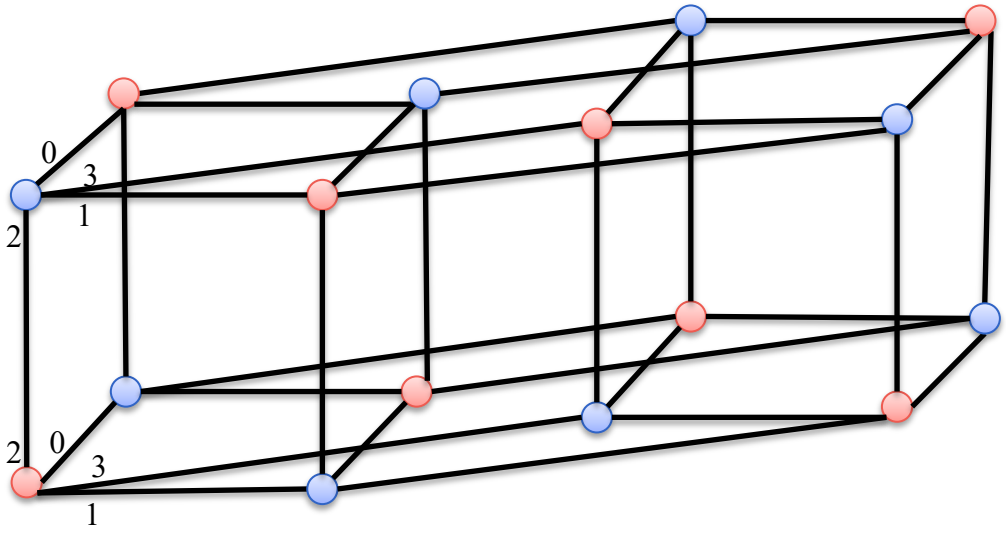
- N noduri de grad $N-1$
- set $direcții = \{1, 2, \ldots, N-1\}$
- condiția de consistență: **direcția de la nodul i la j are sensul $(j-i)\ mod\ N$**



Noțiuni preliminare
Sensul legăturilor (4)

Hipercub:

- pentru un hipercub n dimensional
- set $direcții = \{0, \dots n-1\}$
- condiția de consistență: **două noduri $(b_0, \dots, b_{n-1})$, $(c_0, \dots, c_{n-1})$ legate prin muchie cu eticheta i diferă doar în bitul i**



Notație transmitere mesaje

1. transmitere prin adresare directă

send ch[q] (mesaj)

- unde **q** este identitatea unică, globală a canalului către receptor

2. transmitere prin adresare indirectă

send ch[direcție] (mesaj)

- unde **direcție** este locală procesului care execută operația send
- identifică unul din canalele pe care procesul poate transmite

Algoritm inel

Algoritmul inel

- fiecare proces are un vecin dedicat, Urm
- transmiterea folosește **adresarea prin direcție** (Urm, Prec)
- toate canalele selectate prin Urm formează un ciclu Hamiltonian
- algoritmul este **centralizat**:
  - inițiatorul trimite un token (jeton) care este pasat de fiecare proces de-a lungul ciclului până ajunge înapoi la inițiator;
  - inițiatorul ia apoi decizia

Algoritmul inel

```
chan token[1..n](tok_type tok);
```

```
/* inițiator */
process P[I]{
        tok_type tok;
        send token[Urm](tok);
        receive token[I](tok);
        decide
}
```

```
/* neinițiatori */
process P[k=1 to n, k<>I] {
        tok_type tok;
        receive token[k](tok);
        send token[Urm](tok)
}
```

Număr de mesaje = n, Timp = n

Algoritm arbore

Algoritmul arbore

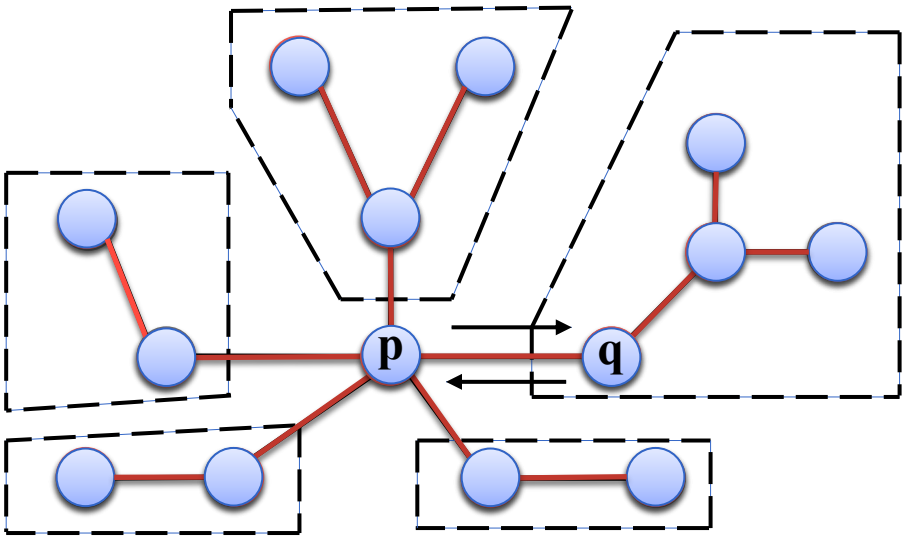
- Se aplică:
  - unei topologii arbore
  - unei topologii arbitrare în care se cunoaște un arbore de acoperire
- Fiecare nod cunoaște identitatea proprie și identitățile vecinilor
- Mulțimea tuturor identităților este **Ids**
- Pentru fiecare proces, se folosesc variabilele locale
  - Vecini – mulțimea identităților vecinilor (**q** = identitatea procesului q)
  - rec[q] – true dacă procesul a primit un mesaj de la vecinul q
- Inițiatorii sunt toate nodurile frunză
- **Algoritm:**
  - fiecare proces trimite exact un mesaj
  - când un proces a primit un mesaj pe fiecare canal incident mai puțin unul (condiție îndeplinită inițial de frunze) el trimite un mesaj pe canalul rămas
  - când un proces a primit câte un mesaj pe toate canalele sale atunci **decide**

Algoritmul arbore (2)

```
chan ch[1:n](int id, tok_type tok);
   /* fiecare proces are un canal propriu */
process Proc[p = 1 to n]{
   bool Vecini[1:n] = vecinii_lui_p;
   bool rec[1:n] = ([|n|] * false);
   int  r = numar_vecini_p;
   tok_type tok; int id, q0;
   while (r > 1){ receive ch[p](id, tok);
        rec[id] = true; r = r - 1 }
   /* de la un singur vecin, q0, nu s-a primit mesaj */
   afla q0 : Vecini[q0] and NOT rec[q0];
   send ch[q0](p, tok);
   receive ch[p](q0, tok); rec[q0] = true;
   decide
   /* informeaza celelalte procese despre decizie */
   /* for [q = 1 to n st Vecini[q] and q<>q0] send ch[q](p, tok); */
}
```

Număr de mesaje = N (egal cu nr. procese), Timp = O(D)

Algoritmul arbore (3)
exemplu de execuție

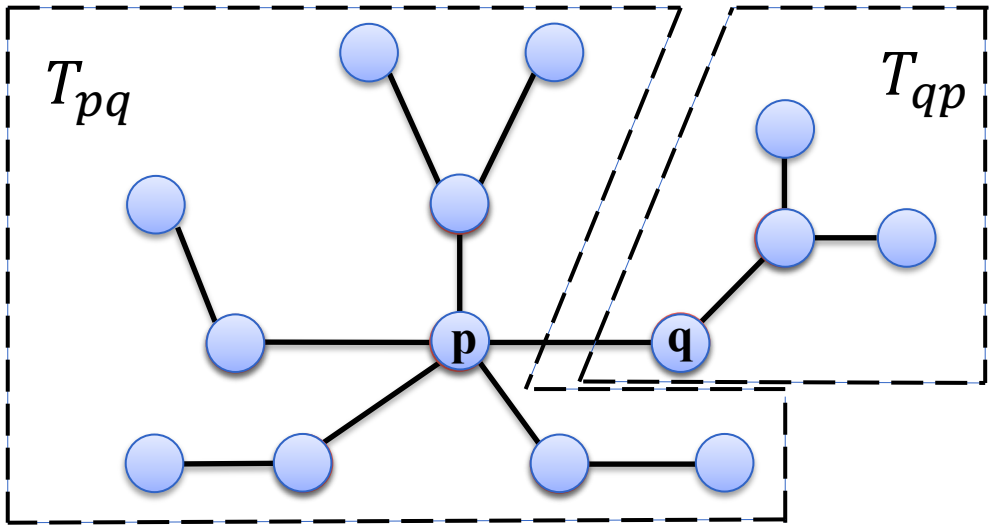


(b) (a) și după recepție, decid
după recepție, decid
transmit

Algoritmul arbore (4)

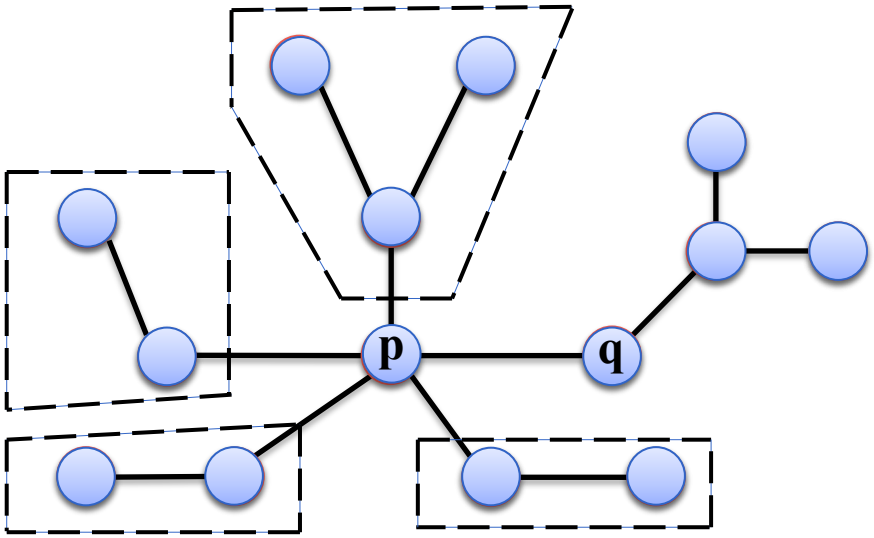
Teoremă: Algoritmul "arbore" este un algoritm undă.

1. calcul finit
   - algoritmul foloseşte cel mult N mesaje → algoritmul atinge o configuraţie terminală γ după un număr finit de paşi.
2. în γ, cel puţin un proces a executat un eveniment *"decide"*
3. *"decide"* este precedat de un eveniment în fiecare proces

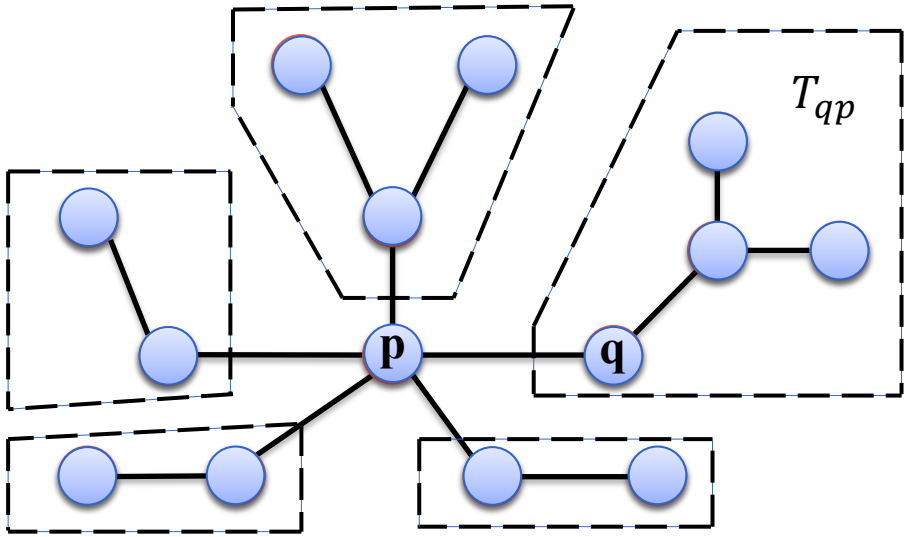


Subseturile T_{pq} şi T_{qp}

Algoritmul arbore (5)



Descompunera lui T_{pq}



Descompunera lui T

Algoritmul arbore

(2) în γ, cel puţin un proces a executat un eveniment "*decide*“

- demo prin reducere la absurd
- N noduri ➔ N-1 legături bidirecţionale între vecini ➔ 2N-2 posibile recepţii de mesaje
- K procese au trimis un mesaj în γ
- mesajele au fost receptionate
- ➔ F = 2N–2–K receptii neconsumate
- Pp. că nici un proces nu a executat "*decide*" în γ
  - fiecare proces este “blocat” la una din operatiile **receive**
- N-K procese nu au trimis mesaj (sunt in bucla while r>1 ...)

  ⇒ fiecare din ele mai are de primit cel puţin 2 mesaje
- K procese au trimis dar nici unul nu a decis

  => fiecare mai are de primit un mesaj

  => receptii neconsumate F >= 2(N-K)+K

Impreuna cu F = 2N–2–K

➔ 2N–2–K >= 2(N-K)+K

-2 >= 0

➔ cel puţin un proces a executat "*decide*" în γ

```
while (r>1){receive ch[p](id,tok);
                    rec[id] = true; r = r-1;}
/* de la un singur vecin, q0, nu s-a primit mesaj */
 afla q0: Vecini[q0] and NOT rec[q0];
 send ch[q0](p, tok);
 receive ch[p](q0, tok); rec[q0] = true;
 decide;
```

Algoritmul arbore

- (3) "decide" este precedat de un eveniment in fiecare proces
- Fie: f_{pq} **transmiterea** unui mesaj de la p la q
  - g_{pq} **receptia** la q a unui mesaj transmis de p
  - Cs submulţimea evenimentelor **e** produse în procesul **s**
  - ≤ relaţia de precedenţă
- Se dem. prin inductie peste evenimentele de recepţie:
- **pentru orice s din Tpq există e ∈ Cs : e ≤ g_{pq}**
- Presupunem că propozitia este adevărată pentru toate evenimentele *receive* care preced g_{pq} şi demonstrăm că este adevărată şi pentru g_{pq}.
- cf. algoritm, p trimite mesaj lui q dupa ce primeste mesaj de la toti ceilalti vecini r
- In Trp (similar in celilalti sub-arbori din Tpq):
- **e ≤ g_{rp} (ipoteza inductie)**
- **g_{rp} ≤ f_{pq}**
- **f_{pq} ≤ g_{pq}**
- **➔ e ≤ g_{pq}**

Algoritm ecou

Algoritmul ecou

- se aplică unor **topologii arbitrare**
- este **centralizat**; există un singur inițiator, I
- propus de Chang; prezentam versiunea mai eficientă Segall
- bazat pe inundarea rețelei cu mesaje **tok**
  - se stabilește un arbore de acoperire
  - mesaje **tok** sunt transmise înapoi spre rădăcină prin canalele arborelui de acoperire

Algoritmul ecou

```
chan ch[1:n](int id, tik_type tok);
const int I = id_initiator;

process Proc(I){
   bool Vecini[1:n] = vecinii_lui_I;
   int r = numar_vecini_I, id;
   tok_type tok;

   for [i=1 to n st Vecini[i]] send ch[i](I, tok);
   while (r > 0) {
       receive ch[I](id, tok);
       r = r - 1;
   }
   decide;
}
```

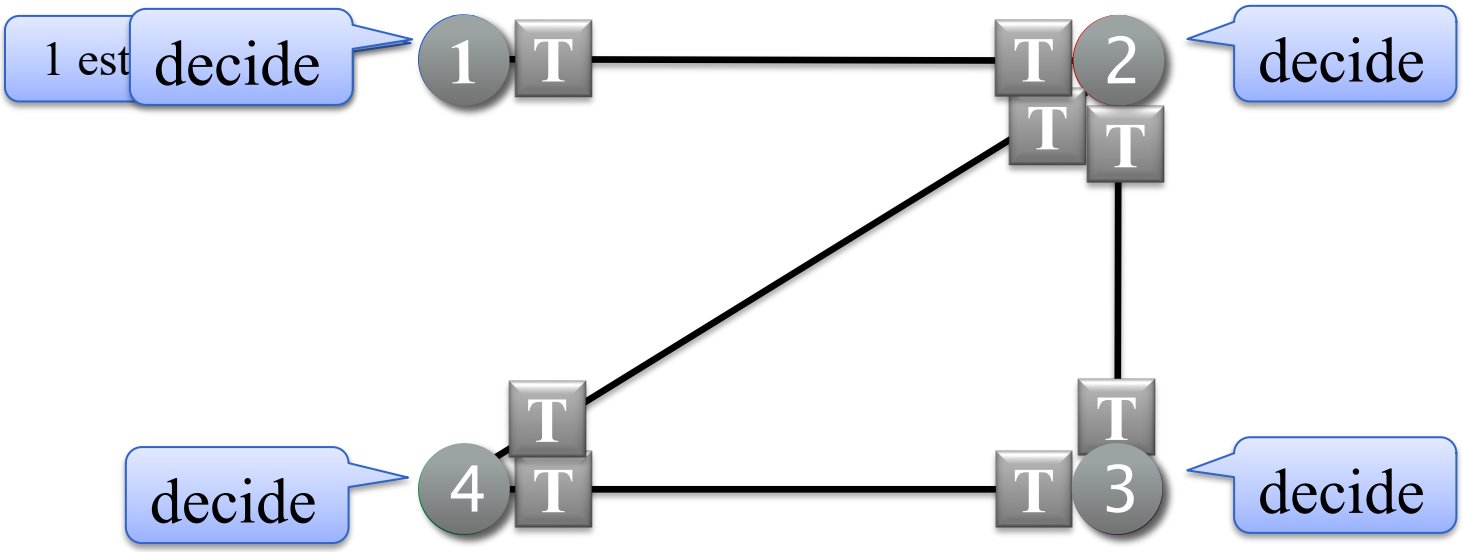
Algoritmul ecou (2)

```
process Proc[p=1 to n, p <> I]{
   bool Vecini[1:n] = vecinii_lui_p;
   int r = numar_vecini_p, id, parinte;
   tok_type tok;

   receive ch[p](parinte, tok); r = r - 1;
   for [q=1 to n st Vecini[q] and q<> parinte)
       send ch[q](p, tok);
   while (r > 0) {
       receive ch[p](id, tok); r = r - 1; }
   send ch[parinte](p, tok)
}
```

$$Mesaje = 2|E|, Timp = O(N)$$

Algoritmul ecou (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Vecini =	{2}	{1, 3, 4}	{2, 4}	{2, 3}
părinte =		1	2	2
r =	0	0	0	0

Algoritmul fazelor
(pulsatiilor, heartbeat, gossiping..)

Algoritmul fazelor

- algoritm **descentralizat**
- **topologii arbitrare**
- **canale unidirecționale**
- vecinii sunt: **in**-vecini și **out**-vecini
- procesele cunosc diametrul grafului D (sau o valoare $D' > D$)
- fiecare proces trimite exact D mesaje fiecărui out-vecin
- mesajul $i + 1$ este trimis fiecărui out-vecin numai după ce i mesaje au fost primite de la fiecare in-vecin

Algoritmul fazelor

```
chan ch[1:n](int id, tok_type tok);
const int D = diametrul_retelei;

process Proc[p = 1 to n] {
   bool in[1:n] = in-vecinii_lui_p;
   bool out[1:n] = out-vecinii_lui_p;
   int rec[1:n] = ([n] 0);
   /* rec[q] = număr mesaje primite de la q */
   int sent = 0;
   /* număr de mesaje transmise fiecărui out-vecin */
   tok_type tok;
   int id;
```

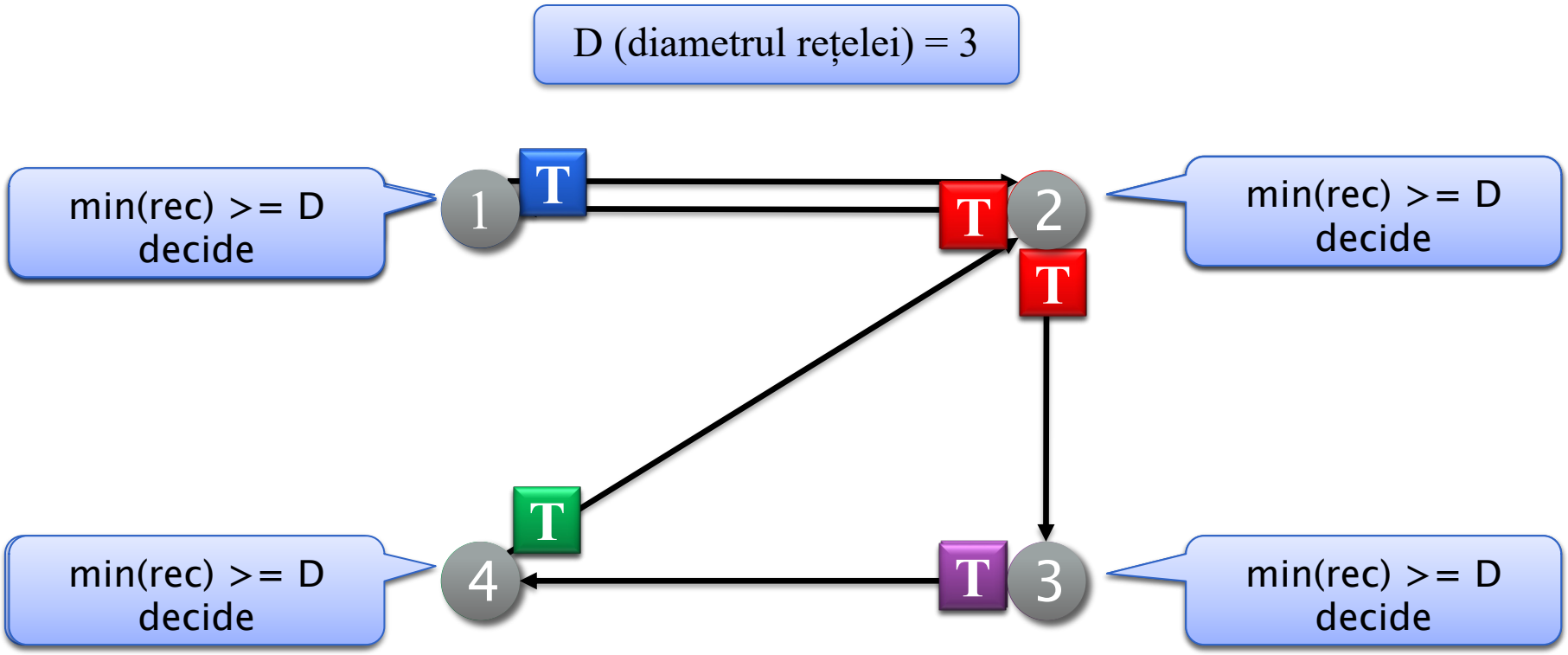
Algoritmul fazelor (2)

```
    if (p este inițiator) {
        for [q = 1 to n st out[q]] send ch[q](p, tok);
        sent = sent + 1
    }

    /* min(rec) este min(rec[q], ∀q ∈ in) */
    while (min(rec) < D) {
        receive ch[p](id, tok); rec[id] = rec[id] + 1;
        if (min(rec) >= sent and sent < D) {
            for [q=1 to n st out[q]] send ch[q](p, tok);
            sent = sent + 1;
        }
    }
    decide;
}
```

Mesaje = 2D|E|, unde E este mulț. canalelor nedirijate (2 canale dirijate)
Timp = 2D

Algoritmul fazelor (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec[4] =	{2:3}	{1:3, 4:3}	{2:3}	{3:3}
sent =	3	3	3	3

Algoritmul fazelor (4)

Teoremă: Algoritmul "fazelor" este un algoritm undă

1. calcul finit
   fiecare proces trimite cel mult D mesaje prin fiecare canal ⇒
   algoritmul atinge o configurație terminală γ după un număr finit de pași

2. în γ, fiecare proces a executat un eveniment "decide"
   presupunem cel puțin un inițiator în C (pot fi mai mulți)
   2.1. fiecare proces a trimis cel puțin un mesaj
   2.2. fiecare proces a decis

3. "decide" este precedat de un eveniment în fiecare proces

- **T. Algoritmul "fazelor" este un algoritm unda**
- (1) **calcul finit**

  fiecare proces trimite cel mult D mesaje fiecarui out-vecin

  → algoritmul atinge o configuratie terminala **γ** intr-un calcul C cu numar finit de pasi
- (2) **in γ, fiecare proces a inregistrat un eveniment "decide"**

  presupunem cel putin un initiator in calculul C (pot fi mai multi)

  (2.1) fiecare proces a trimis cel putin un mesaj

```
if (p este initiator){
   for [q = 1 to n st out[q]]{
     send ch[q](p, tok); sent = sent+1;}

while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
  ...
```

daca p ne-initiator: **sent = 0** → cand p primeste primul mesaj, transmite unul tuturor out-vecinilor

(2.2) In configuratia terminala γ fiecare proces a decis

```
while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
} decide;
```

Demonstram ca orice proces iese din ciclu, adica min(rec) = D , deci nu exista in-vecin care sa-i fi trimis mai putin de D mesaje

Fie p procesul cu cel mai mic $sent_p$ in γ ➔ ∀q $sent_q >= sent_p$ (1)

In particular, se aplica pentru toti in-vecinii q ai lui p

In γ, toate mes. trimise de q sunt recept. de p ➔ $sent_q = rec_p[q]$ (2)

(1)&(2) → $min_q (rec_p[q]) >= sent_p$ conditia din **if** este semi-indeplinita

➔ $sent_p$ =D; altfel p ar fi trimis mesaje suplimentare la ultima receptie

$sent_q$ =D ∀ q ➔ $rec_p[q]$ =D pentru orice canal qp

min(rec) = D ➔ fiecare proces a iesit din ciclu si a decis

- **(3) "decide" este precedat de un eveniment in fiecare proces**

  Ne referim la decizia din procesul **p**

  D este diametrul → pentru orice **q** exista o cale la **p** cu lungime <= D

  q = p(0), p(1), …, p(k) = **p**, cu lungimea k<=D

  Intr-o faza, transmisia precede receptia aceluiasi mesaj

  $f_{p(i)p(i+1)}^{(i+1)} \leq g_{p(i)p(i+1)}^{(i+1)}$ pentru 0<= i <k

  Din algoritm, receptia unui mesaj precede transmiterea lui in faza urmatoare

  $g_{p(i)p(i+1)}^{(i+1)} \leq f_{p(i+1)p(i+2)}^{(i+2)}$ pentru 0<= i <k-1

  Aplicand succesiv relatiile, rezulta $f_{p(0)p(1)}^{(1)} \leq g_{p(k-1)p(k)}^{(k)}$

  In plus, din algoritm, $g_{p(k-1)p(k)}^{(k)} \leq d_p$

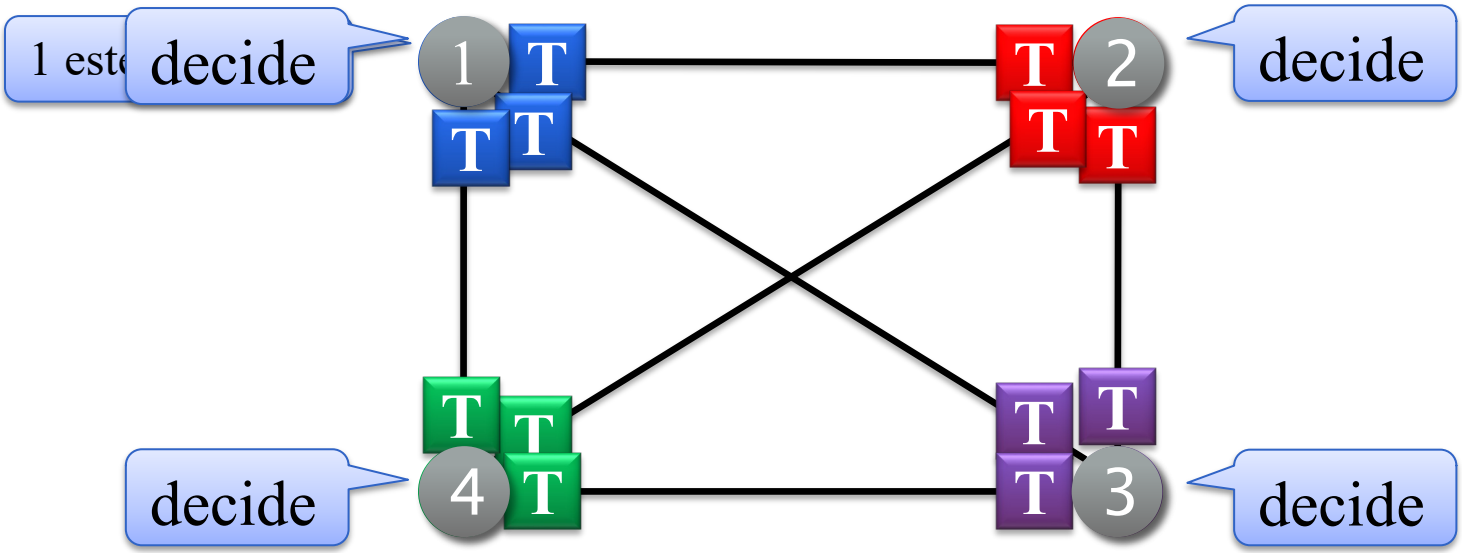
  ➔ decizia lui p este precedata de un eveniment in orice q: $f_{qp(1)}^{(1)} \leq d_p$

Algoritmul fazelor pentru clici

```
chan ch[1:n](int id, tok_type tok);
process Proc[p = 1 to N] {
   bool Vecini[1:n] = vecinii_lui_p;
   int n_vecini = numar-vecini_p;
   int rec = 0, sent = 0, id;
   tok_type tok;
   if (p este initiator) {
        for [q=1 to N st Vecini[q]] send ch[q](p, tok);
        sent = sent+1;
   }
   while (rec < n_vecini) {
        receive ch[p](id, tok); rec = rec + 1;
        if (sent == 0) {
              for [q=1 to N st Vecini[q]] send ch[q](p, tok);
              sent = sent + 1;
   }    }
   decide;
}
```

$$Mesaje = N(N-1), Timp = 2$$

Algoritmul fazelor pentru clici (2) exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec =	3	3	3	3
sent =	1	1	1	1

Algoritmul lui Finn

Algoritmul lui Finn

- nu cere cunoașterea diametrului
- se bazează pe cunoașterea identificatorilor proceselor
- în mesaje se transmit seturi de identificatori de procese
- proc **p** păstreaza două mulțimi de ids

  Inc – mulțimea proceselor **q** pentru care un eveniment în **q** precede cel mai recent eveniment în **p**

  NInc – mulțimea proceselor **q** pentru care fiecare vecin **r** are un eveniment care precede cel mai recent eveniment în **p**

- **Algoritmul:**

  - inițial: $Inc = \{p\}, \quad NInc = \emptyset$

  - p trimite mesaje cu Inc și $NInc$ de fiecare dată când Inc sau $NInc$ crește

  - când p primește mesaje cu Inc și $\mathrm{N}Inc$, actualizează Inc și $\mathrm{N}Inc$ (reuniune)

  - când p a primit un mesaj de la toți in-vecini, p este inserat în $\mathrm{N}Inc$

  - când Inc devine egal cu $\mathrm{N}Inc$, p decide

Algoritmul lui Finn (2)

```
typedef SOP {multime identificatori};
chan ch[1:N](int id, SOP Inc, SOP NInc);

process Proc[p = 1 to n]{
   SOP Inc = {p};
   SOP Ninc = Ø;
   /* de la cine a recepționat */
   bool rec[1:n] = ([n] false);
   bool out[1:n] = out-vecinii_lui_p;
   bool in[1:n] = in-vecinii_lui_p;
   SOP rInc, rNInc;
   int id;

   if (p este initiator)
       for [q = 1 to n st out[q]]
          send ch[q](p, Inc, NInc);
```

Algoritmul lui Finn (2)

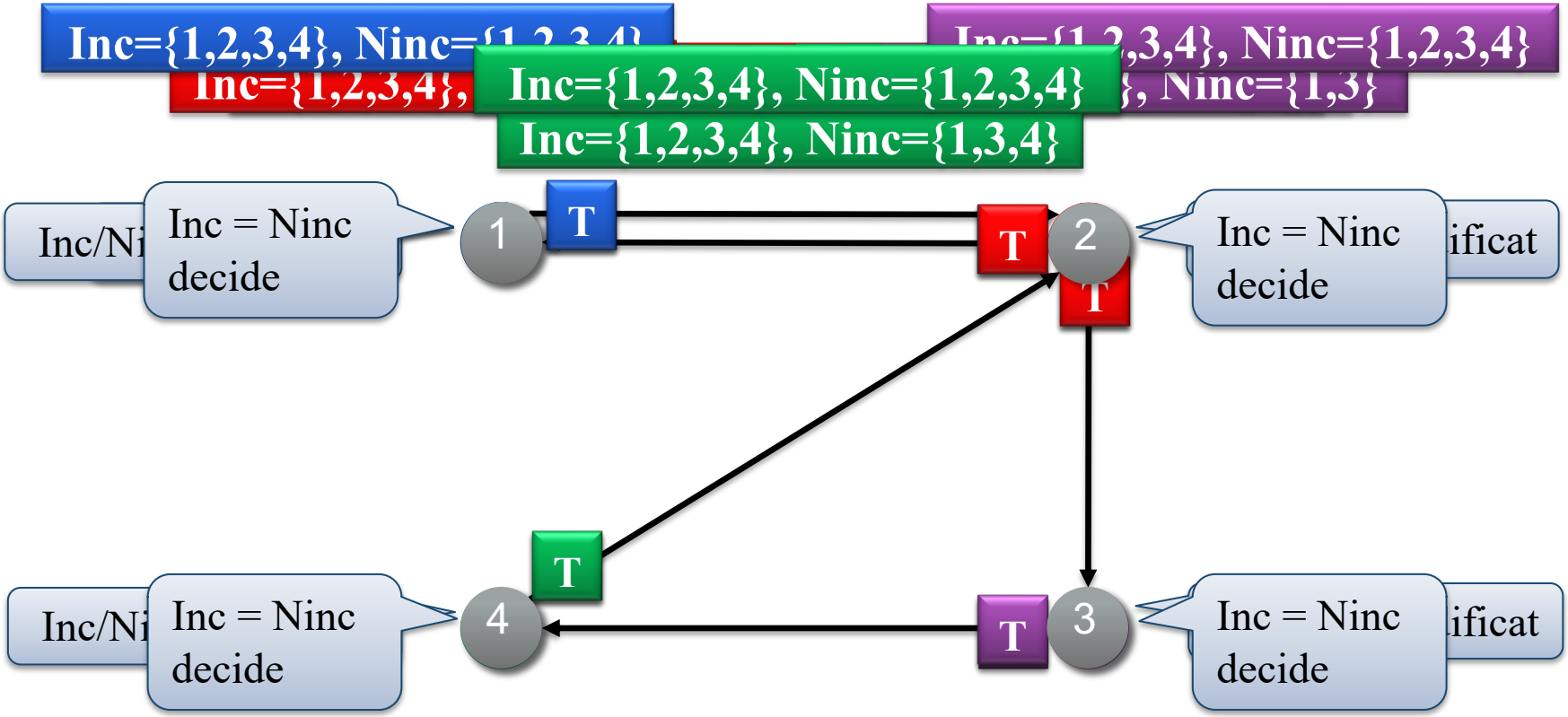
```
  while (Inc <> Ninc) {
      receive ch[p](id, rInc, rNInc);

      Inc  = Inc  U rInc;
      NInc = NInc U rNInc;

      rec[id] = true;
      if (rec[id], for ∀id=1..n and in[id])
            NInc = NInc U {p};
      if (Inc sau NInc modificat)
            for [q=1 to n st out[q]]
                  send ch[q](p, Inc, NInc);
  }
  decide;
}
```

$$Mesaje \leq 2N|E|, \qquad Timp = O(D)$$

Algoritmul lui Finn (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Inc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
Ninc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
rec =	{2:T}	{1:T, 4:T}	{2:T}	{3:T}

Algoritmul lui Finn (4)

Teoremă: Algoritmul lui Finn este un algoritm undă

1. calcul finit

2. în γ, fiecare proces a executat un eveniment "decide“

   2.1. în γ fiecare proces a trimis cel puțin un mesaj pe fiecare canal (dem. similară alg. fazelor)

   2.2. în γ fiecare proces a decis

      2.2.1. $\forall p,\ Inc_p\ conține\ toate\ procesele\ (în\ \gamma)$

      2.2.2. $\forall p\ și\ q,\ NInc_p = NInc_q$

3. "decide" este precedat de un eveniment în fiecare proces

T. Algoritmul lui Finn este un algoritm unda

- (1) **calcul finit**
- Se trimite un mesaj la fiecare modificare Inc sau Ninc
- Dimensiunea celor doua seturi luate impreuna este initial 1 si ajunge la 2N
  → numarul de mesaje transmise este limitat superior la 4N|E|

- (2) **In γ, fiecare proces a executat un eveniment "decide"**
- 2.1. in **γ** fiecare proces a trimis cel putin un mesaj pe fiecare canal:

  p trimite tuturor vecinilor q → se schimba Inc_q → q transmite vecinilor sai; in plus graful este tare conex etc.
- 2.2. in **γ** fiecare proces a decis

  2.2.1. ∀ p, Inc_p contine toate procesele (in γ)

  daca exista arc pq, la modificare Inc_p, p il trimite lui q; q il include in Inc_q

  graf tare conex → pentru orice p si q: Inc_q include Inc_p

  impreuna cu Inc_p include Inc_q → $Inc_p = Inc_q$ pentru **∀** p si q

  → Inc_p contine identitatile tuturor proceselor

2.2.2. ∀ p si q, $NInc_p = NInc_q$

Fiecare proces a trimis un mesaj pe fiecare canal

→ fiecare proces a receptionat de la toti vecinii (∀ id € in: rec[id])

→ (cf alg) p este adaugat la NInc

$NInc_p$ contine toate procesele – demo similara cu Inc_p

→ $Inc_p = NInc_p$ in γ

→ fiecare proces a decis in γ

- (3) In procesul p, "decide" este precedat de un eveniment in fiecare proces

  Fie q un in-vecin al lui p

  Fie evenimentul intern lui q, **a_q: NInc = NInc U {q}**

  conform algoritmului, a_p se executa daca pentru fiecare in-vecin r al lui q un eveniment din r precede a_q

  Procesul p decide doar cand $Inc_p = NInc_p$ = setul tuturor proceselor

→ cand procesul p ia decizia:

- p € Inc
- in-vecinul q € Inc si q € NInc
- deci decizia lui p este precedata de un eveniment din q si un eveniment din fiecare din in-vecinii lui q
- relatia este valabila pentru oricare doua procese din care unul este in-vecinul celuilalt
- retea conexa → decizia lui p este precedata de un eveniment din fiecare din celelalte procese

Sumar

- Sisteme de tranziții
- Caracteristici ale algoritmilor undă
- Algoritmul inel
- Algoritmul arbore
- Algoritmul ecou
- Algoritmul fazelor
- Algoritmul lui Finn